ChatGPT

MCA: Quests 1.5.4 — Compatibility, Objective Guidance, and Integration Specification

Document status: Implementation specification
Target release: MCA: Quests `1.5.4`
Audience: Coding agent / maintainer implementing the release
Minecraft version: Unspecified by product requirement. The current MCA: Quests repository is concretely based on Minecraft `1.20.1`, Forge `47.4.10+`, MCA Reborn `7.6.x`, Architectury API, and JDK 17, so those versions are the reference implementation baseline, not a new cross-version promise. [1]
Primary compatibility baselines researched: Ice & Fire: Community Edition `1.20.1` source branch plus its current documentation; Bountiful `1.20.1` source branch plus current Bountiful/Kambrik documentation. Ice & Fire CE explicitly says its `1.20.1` and `1.21.1` code lines have diverged substantially and that `1.20.1` now receives bug fixes rather than feature backports, so compatibility must be capability-based rather than assuming parity across Minecraft versions. [2]

Executive summary and scope

MCA: Quests already has the right architectural precedents for this release: server-authoritative quest state, datapack-driven content, runtime resolution for fragile optional dependencies, explicit capability reporting, suspend/resume behavior when optional integrations disappear, and isolated compatibility bridges. The current repository documents those patterns for MCA itself, FTB Quests, MCA: Conversations, and Townstead. In particular, Townstead compatibility deliberately avoids static type references, reports individual capabilities rather than one binary “installed” flag, and suspends dependent active quests without destroying their progress when the integration becomes unavailable. [3]

Version `1.5.4` should apply that model in three places:

Theme	1.5.4 outcome
Ice & Fire: Community Edition	Detect original Ice & Fire versus CE despite their shared `iceandfire` namespace/mod identity; remove assumptions tied to the original Java packages; capability-probe registries; support CE-only seekers/netherite content; suppress or suspend unavailable Myrmex content; validate every integration by registry ID rather than class identity.
Objective indicator	Treat the already-existing 1.5.3 edge indicator as a subsystem to harden, not rewrite. Separate projection from temporal stabilization, handle near-plane/behind-camera degeneracy correctly, introduce edge-state hysteresis and frame-rate-independent smoothing, guarantee safe-rectangle placement, reset on discontinuities, and keep the normal path free of raycasts and allocations.

Theme	1.5.4 outcome
Bountiful	Make the primary integration **data/API boundary first**, not direct coupling to Bountiful internals. Ship Bountiful pool/decree additions, add an MCA-side compatibility SPI, optionally hook successful bounty cash-in for an MCA `bountiful_bounties` objective, expose board-interaction UX, and degrade cleanly to data-only mode if the completion hook cannot bind.

This is not a loader rewrite. The current MCA: Quests artifact is Forge-oriented even though Architectury is present through MCA and other researched integrations are multi-loader. [4] The 1.5.4 implementation **MUST** put loader- or event-specific behavior behind small platform interfaces so a Fabric/NeoForge port does not require redesigning compatibility logic, but it **MUST NOT** claim Fabric/NeoForge support until a build and test matrix for those loaders actually exists.

Primary assumptions and explicit non-assumptions

Area	Specification assumption
MCA: Quests source	Implement against the current `main` architecture unless the release branch differs. Existing public datapack/API contracts are preserved.
Minecraft	Product target is unspecified; `1.20.1 + Forge 47.4.10 + JDK 17` is the concrete reference baseline found in the repository. [1]
Ice & Fire CE	`1.20.1` is the exact source baseline used for IDs below. A later-version port must rediscover capabilities from registries and manifests rather than hardcode “CE version $\geq$ X”.
Bountiful	Its documented extension surface is primarily pools/decrees/data plus its own internal Kotlin model. No stable third-party “bounty completed” callback API was found in the primary documentation/source reviewed; therefore completion tracking needs an MCA-owned optional shim, not an assumption that such an API exists. Bountiful's source exposes built-in bounty logic for item, item tag, entity, command and criteria types.
Saves	Existing quest definitions, quest IDs, player progress, frozen objective/reward rolls, projects, situations and compatibility state must continue to load without destructive migration.
Networking	Avoid a protocol change for these features. The edge indicator is client-derived from existing guidance data; CE and Bountiful progress can remain server-side. Add a packet only if implementation proves impossible without one.
Optional mods	MCA: Quests must start and function normally with neither I&F nor Bountiful installed. No normal classloader path may load an optional-mod class.

Release-level acceptance criteria

`1.5.4` is complete when the following are all true:

1. Installing I&F CE no longer crashes or breaks MCA: Quests because code expects original Ice & Fire package names.
2. CE-common dragon/mob quests work by registry ID; CE-exclusive content can be used by gated quests; Myrmex-dependent content is never offered under CE and existing active Myrmex objectives suspend instead of becoming corrupt or impossible.
3. An off-screen target cannot produce a NaN/Infinity HUD position, a marker outside the configured safe rectangle, a persistent stale arrow, or rapid edge/on-screen flicker from subpixel camera motion.
4. Bountiful can coexist without code linkage; its data extension works independently of the completion hook; successful Bountiful cash-ins can advance explicitly configured MCA objectives when the hook capability is available.
5. Removing either optional mod from an existing world does not delete dependent quest progress.
6. Existing 1.5.x quest JSON remains valid without modification.
7. Dedicated-server startup verifies that no client marker class, I&F implementation class, or Bountiful implementation class is eagerly loaded.
8. Unit tests, save-upgrade fixtures, datapack validation, localization parity checks, integration smoke tests, and the build all pass.

Prioritized feature list

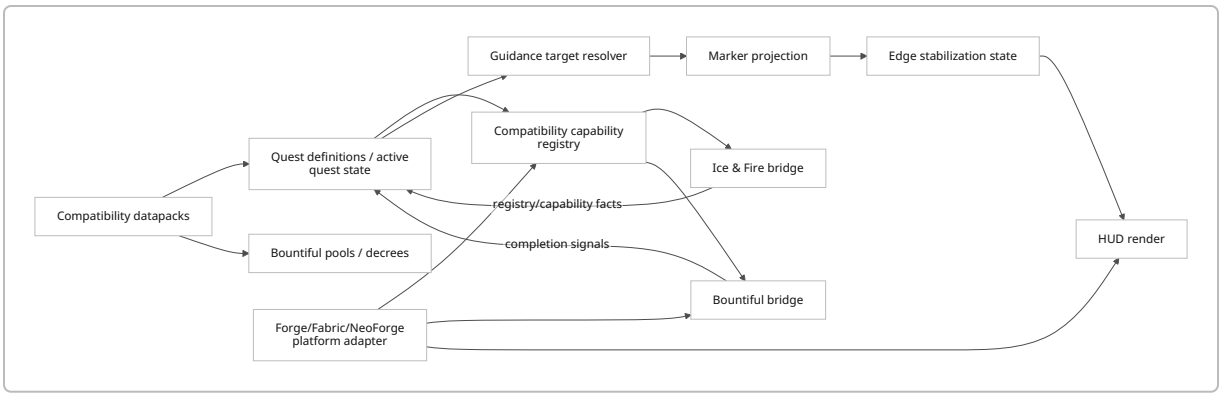
Priority	Feature	Effort	Risk	Reason
P0	Generic optional-mod capability framework used by I&F/Bountiful	Medium	Medium	Prevents two new one-off compatibility architectures.
P0	I&F CE flavor detection + registry capability probing	Medium	Medium	Core crash/data-integrity prerequisite.
P0	I&F CE content corrections and Myrmex gating	Medium	Medium	Makes existing and new quest content usable.
P0	Edge-indicator projection hardening, hysteresis, smoothing	Medium	Medium	Direct reliability goal; localized client work.
P0	Bountiful data-pack/pool integration	Medium	Low	Uses Bountiful's intended data-driven customization route. [5]
P1	Bountiful successful-cash-in compatibility hook	High	High	No public completion-event contract was found; therefore this must be carefully version/capability guarded.
P1	`bountiful_bounties` MCA objective + board-interaction objective	Medium	Medium	Creates meaningful UX beyond mere coexistence.

Priority	Feature	Effort	Risk	Reason
P1	Conditional compatibility-pack registrar	Medium	Medium	Prevents dangling I&F/Bountiful content when one side is absent.
P1	`/mcaquests compat ...` diagnostics and validation	Medium	Low	Essential for maintaining reflection/mixin/data integrations.
P1	Localization/config/migration additions	Medium	Low	Required release quality.
P2	Optional “off-screen or occluded” arrow mode	Medium	Medium	Useful enhancement but introduces occlusion sampling.
P2	Broader CE-themed built-in narrative quest set	High	Medium	Valuable content, but secondary to compatibility correctness.
P2	Full Fabric/NeoForge build targets	High	High	Architecture should allow this; 1.5.4 need not make an unsupported release promise.

The recommended compatibility philosophy is deliberately consistent with MCA: Quests itself: the repository already treats fragile integrations through isolated adapters and capabilities, while Bountiful's own content architecture is driven by data pools/decrees and worth balancing. [6]

Comparable system	Useful behavior	Lesson for 1.5.4	Recommended adoption
MCA: Quests ↔ Townstead	Runtime resolution, granular capabilities, suspend/resume dependent quests. [3]	Optional integrations should fail locally rather than disable the quest engine.	**Adopt directly** as compatibility model.
MCA: Quests ↔ FTB Quests	Explicit bridge/no-op behavior and no required FTB runtime. [3]	Keep optional-mod-facing code behind one seam.	**Adopt** for Bountiful bridge.
Bountiful	Pools, decrees, randomized amounts, worth matching, board reputation. [7]	Do not try to replace Bountiful's economy with MCA difficulty math.	**Integrate at data boundary**.
Bountiful Criteria	Reuses advancement criteria but excludes high-frequency `tick` / `enter_block`; criteria have no historical “memory.” [8]	Cross-system conversion is not semantically lossless.	**Map only safe types automatically**.

Comparable system	Useful behavior	Lesson for 1.5.4	Recommended adoption
Existing MCA marker system	World guidance plus an edge indicator, frame-stamped HUD transfer and configurable occlusion.	Reliability work should preserve renderer/HUD separation.	**Refine rather than replace**.



Ice & Fire: Community Edition integration

Ice & Fire CE is not merely a drop-in package-renamed build. Its official README describes it as an unofficial fork with optimizations, new content and rewrites, explicitly omits the entire Myrmex series, and warns users not to directly replace the original mod because doing so can corrupt saves. The project also states that `1.20.1` and `1.21.1` have substantially diverged. [2] That makes version-string matching or “mod id present = compatible” an inadequate integration strategy.

A recent CE issue demonstrates the practical package-linkage failure mode: another add-on referencing the original `com.github.alexthe666.iceandfire...IafItemRegistry` class crashes under CE because CE reorganized implementation classes under `com.iafenvoy.iceandfire`. [9] MCA: Quests **MUST NOT** repeat this pattern.

Required architecture

Create:

```
dev.otectus.mcaquests.compat
├── CompatProvider.java
├── CompatCapability.java
├── CompatStatus.java
├── CompatRegistry.java
└── iceandfire
    ├── IceAndFireCompat.java
    ├── IceAndFireFlavor.java
    ├── IceAndFireCapabilities.java
```

```
    ├── IceAndFireRegistryManifest.java
    └── IceAndFireDiagnostics.java
```

Suggested contracts:

```
public interface CompatProvider {
    String id();
    CompatStatus status();
    Set<String> capabilities();
    void reprobe();
}

public enum IceAndFireFlavor {
    NONE,
    ORIGINAL,
    COMMUNITY_EDITION,
    AMBIGUOUS
}

public record IceAndFireStatus(
        IceAndFireFlavor flavor,
        String detectedVersion,
        Set<String> capabilities,
        Map<String, String> missingRequirements
) {}
```

Do **not** expose `com.iafenvoy.*` or `com.github.alexthe666.*` types in a signature, generic parameter, field descriptor, annotation, mixin interface or class literal.

The CE documentation explicitly recommends distinguishing implementations by class existence: the original root is under `com.github.alexthe666.iceandfire`, whereas CE uses `com.iafenvoy.iceandfire`. [9] Use that only to identify the flavor; use vanilla registries to determine actual feature support.

```
static IceAndFireFlavor detectFlavor(ClassLoader loader) {
    boolean ce = isPresent(loader, "com.iafenvoy.iceandfire.IceAndFire");
    boolean original = isPresent(loader,
"com.github.alexthe666.iceandfire.IceAndFire");

    if (ce && original) return IceAndFireFlavor.AMBIGUOUS;
    if (ce) return IceAndFireFlavor.COMMUNITY_EDITION;
    if (original) return IceAndFireFlavor.ORIGINAL;
    return IceAndFireFlavor.NONE;
}
```

```
static boolean entityExists(ResourceLocation id) {
    return BuiltInRegistries.ENTITY_TYPE.containsKey(id);
}

static boolean itemExists(ResourceLocation id) {
    return BuiltInRegistries.ITEM.containsKey(id);
}
```

`AMBIGUOUS` **MUST** produce a warning and registry-only operation. It must not arbitrarily select one mod's internal API.

Capability model

Recommended initial capabilities:

```
iceandfire.core
iceandfire.fire_dragon
iceandfire.ice_dragon
iceandfire.lightning_dragon
iceandfire.myrmex
iceandfire.dread_mobs
iceandfire.dragon_seekers
iceandfire.netherite_dragon_armor
iceandfire.netherite_hippogryph_armor
iceandfire.brush_scales
iceandfire.dragon_forge_blood
```

Capabilities derived purely from registry IDs should be marked `REGISTRY_CONFIRMED`. Behavioral capabilities that cannot be proven from registries alone should be `FLAVOR_DECLARED` or `ADAPTER_CONFIRMED`, allowing diagnostics to tell the difference.

For example:

```
capabilities.set("iceandfire.dragon_seekers",
    itemExists(id("iceandfire:dragon_seeker"))
        && itemExists(id("iceandfire:epic_dragon_seeker"))
        && itemExists(id("iceandfire:legendary_dragon_seeker"))
);

capabilities.set("iceandfire.myrmex",
    entityExists(id("iceandfire:myrmex_worker"))
        || entityExists(id("iceandfire:myrmex_queen"))
);
```

The latter deliberately does **not** say “CE means no Myrmex.” It asks the game registry what is actually present. The official CE project currently says Myrmex is omitted, but capability probing also survives a future CE backport/add-on that restores one. [10]

Quest-relevant CE entity mapping

The CE `1.20.1` registry source contains fire, ice and lightning dragons plus the broader mythical/Dread roster, and contains no Myrmex entity registrations. The exact registered names below come from `IafEntities`.

Category	Quest-safe entity IDs	Default MCA use
Dragons	`iceandfire:fire_dragon`, `iceandfire:ice_dragon`, `iceandfire:lightning_dragon`	Kill, locate/guidance, themed chains
Flying/ tameable creatures	`iceandfire:hippogryph`, `iceandfire:amphithere`	Kill only where thematically appropriate; otherwise observe/tame-style quest if supported
Ocean	`iceandfire:siren`, `iceandfire:hippocampus`, `iceandfire:sea_serpent`	Kill/visit/gather
Mythic hostile/ neutral	`iceandfire:gorgon`, `iceandfire:cyclops`, `iceandfire:deathworm`, `iceandfire:cockatrice`, `iceandfire:stymphalian_bird`, `iceandfire:troll`, `iceandfire:hydra`, `iceandfire:ghost`	Kill/boss hunt
Pixie	`iceandfire:pixie`	Prefer non-lethal/ gather content unless explicitly intended
Dread	`iceandfire:dread_thrall`, `iceandfire:dread_ghoul`, `iceandfire:dread_beast`, `iceandfire:dread_scuttler`, `iceandfire:dread_lich`, `iceandfire:dread_knight`, `iceandfire:dread_horse`	Kill/hunt
Physical/ special entity	`iceandfire:stone_statue`	Normally exclude from generic kill pools

The registry also defines technical/projectile/part IDs that **MUST NOT** be put into generic “kill a mob” compatibility pools:

```
iceandfire:dragon_multipart
iceandfire:multipart
```

```
iceandfire:hydra_multipart
iceandfire:cylcops_multipart    # exact upstream spelling
iceandfire:dragon_egg
iceandfire:dragon_arrow
iceandfire:dragon_skull
iceandfire:fire_dragon_charge
iceandfire:ice_dragon_charge
iceandfire:lightning_dragon_charge
iceandfire:hippogryph_egg
iceandfire:deathworm_egg
iceandfire:cockatrice_egg
iceandfire:stymphalian_feather
iceandfire:stymphalian_arrow
iceandfire:amphithere_arrow
iceandfire:sea_serpent_bubbles
iceandfire:sea_serpent_arrow
iceandfire:chain_tie
iceandfire:pixie_charge
iceandfire:tide_trident
iceandfire:mob_skull
iceandfire:dread_lich_skull
iceandfire:hydra_breath
iceandfire:hydra_arrow
iceandfire:ghost_sword
```

Those names are exact `1.20.1` CE registry identifiers; notably, the multipart Cyclops registry key is spelled `cylcops_multipart` upstream and must not be silently “corrected” in code that audits registries.

CE-exclusive/new-content mapping

The current CE documentation identifies Netherite Dragon Armor, four Dragon Seeker tiers, brushing dragons to obtain scales, Dragon Forge blood crafting, and other new integrations/content. The seekers have radii/semantics of 150 blocks for normal, 200 for alive epic, 300 for alive untamed legendary, and 500 for creative-only godly. [11]

The `1.20.1` source registers the seeker item names directly. Netherite is an explicit dragon-armor material in CE source, and the localized item keys confirm the four generated part IDs.

CE content	Exact registry IDs / representation	Integration action
Normal Dragon Seeker	`iceandfire:dragon_seeker`	Gate CE quest; gather/ craft/use-item objective
Epic Dragon Seeker	`iceandfire:epic_dragon_seeker`	Same

CE content	Exact registry IDs / representation	Integration action
Legendary Dragon Seeker	`iceandfire:legendary_dragon_seeker`	Same
Godly Dragon Seeker	`iceandfire:godly_dragon_seeker`	**Never** normal survival requirement; creative/admin content only
Netherite Dragon Armor	`iceandfire:dragonarmor_netherite_head`, `..._neck`, `..._body`, `..._tail`	Gather/craft/upgrading quest
Netherite Hippogryph Armor	`iceandfire:netherite_hippogryph_armor`	Optional gather/equipment quest after registry probe
Brush scales behavior	Existing scale items; behavior is new acquisition route	Avoid special dependency: existing gather objectives naturally accept scales acquired this way
Forge blood behavior	Existing blood result becomes craftable via CE's Dragon Forge route	Existing gather/craft outcome can work without reflecting into forge code

For seeker **use**, introduce a reusable generic objective rather than an I&F-specific objective:

```
{
  "type": "mcaquests:use_item",
  "item": "iceandfire:legendary_dragon_seeker",
  "count": 1,
  "require_success": false
}
```

Recommended event abstractions:

```
public interface PlayerUseItemHooks {
    AutoCloseable listen(BiConsumer<ServerPlayer, ItemStack> listener);
}
```

Forge implementation can use the appropriate right-click/use-item event; a Fabric implementation can bind the corresponding item-use callback. The **common quest engine must not know which loader emitted the event**.

`require_success=false` is intentional: MCA: Quests can reliably prove that the player attempted to use the registered item but, without coupling to CE's internal seeker implementation, cannot reliably prove that the seeker actually found a dragon. Do not manufacture that semantic guarantee.

Myrmex correction and active-quest behavior

CE explicitly removes the Myrmex series. [2] Therefore:

```
Offer generation:
  required capability missing -> omit quest from candidate set

Reload:
  active objective target missing -> state = SUSPENDED_COMPAT

Turn-in:
  suspended quest -> cannot complete, can always abandon

Capability restored:
  restore ACTIVE state
  retain objective counts, frozen targets, rolled rewards and timestamps
```

Add a generic condition rather than hard-wiring CE into content:

```
{
  "type": "mcaquests:compat_capability",
  "provider": "iceandfire",
  "capability": "iceandfire.myrmex",
  "present": true
}
```

and:

```
{
  "type": "mcaquests:compat_capability",
  "provider": "iceandfire",
  "capability": "iceandfire.dragon_seekers",
  "present": true
}
```

This condition becomes reusable for Bountiful and future compatibility modules.

Worldgen/structure behavior

CE says biome configuration is controlled through datapack biome tags and that the fork contains rewrites; the 1.20.1/1.21.1 lines are materially different. [10] Consequently, MCA: Quests **MUST NOT** reflect into original Ice & Fire biome/worldgen classes to find quest destinations.

For any I&F structure objective:

```
1. Resolve requested structure ResourceLocation through Minecraft's registry.
2. If absent -> capability unavailable / quest gated.
3. If present -> use vanilla/loader-supported locate mechanisms.
4. Never force-load arbitrary chunks each tick.
5. Cache a successful locate result into the accepted quest's frozen target
data.
```

Versioned manifest

Store expected content as data, not code branches:

```
{
  "schema": 1,
  "provider": "iceandfire",
  "flavor": "community_edition",
  "minecraft": "1.20.1",
  "entities": {
    "dragons": [
      "iceandfire:fire_dragon",
      "iceandfire:ice_dragon",
      "iceandfire:lightning_dragon"
    ],
    "excluded_technical": [
      "iceandfire:dragon_multipart",
      "iceandfire:hydra_multipart"
    ]
  },
  "items": {
    "dragon_seekers": [
      "iceandfire:dragon_seeker",
      "iceandfire:epic_dragon_seeker",
      "iceandfire:legendary_dragon_seeker",
      "iceandfire:godly_dragon_seeker"
    ]
  }
}
```

The manifest is an **expectation/diagnostic catalog**, never an assertion that an ID exists. Actual registry probing remains authoritative.

Required compatibility fixes

Existing assumption to remove	Fix
`iceandfire` loaded means original I&F	Flavor probe plus registry capability map
Direct original I&F class imports	Eliminate; build-time constant-pool tripwire
Myrmex content available whenever `iceandfire` exists	Gate by specific Myrmex registry capability
Feature support inferred from I&F version	Probe item/entity/structure registries
Worldgen internals stable	Use vanilla registries/locate APIs
CE replacing original is a routine migration	Do not claim this; CE itself warns direct replacement can corrupt saves. [10]
Every I&F entity is a killable quest target	Maintain quest-safe allowlists; exclude parts/projectiles
CE `1.21.1` matches `1.20.1`	Separate manifests/probes; upstream explicitly says code differs. [10]

Objective indicator reliability design

MCA: Quests already contains the fundamental edge-indicator pipeline. `QuestMarkerRenderer` projects a target and publishes edge state into `MarkerFrameState`; `QuestMarkerHud` consumes that state in the GUI render phase rather than trying to reconstruct the projection matrix there. `MarkerFrameState` includes stale-frame protection specifically so a frame in which the world renderer never runs cannot leave an obsolete marker painted forever. The current config already has `questMarkerEdgeIndicator` and occlusion choices.

The 1.5.4 work therefore **MUST preserve** this architecture and replace only the raw edge-decision/stabilization layer.

Proposed module split

```
client/marker/
├── MarkerProjection.java          # stateless projection math
├── EdgeDirection.java             # normalized screen-direction result
├── EdgeSafeRect.java              # geometry/inset calculations
├── EdgeIndicatorState.java        # temporal state, hysteresis, resets
├── EdgeIndicatorSmoother.java     # frame-time-independent filter
├── MarkerOcclusionSampler.java    # optional P2 CPU raycast policy
├── MarkerFrameState.java          # existing world->HUD transfer
├── QuestMarkerRenderer.java       # existing orchestration
└── QuestMarkerHud.java            # existing HUD drawing
```

`MarkerProjection` stays pure and unit-testable. `EdgeIndicatorState` is the **only** stateful projection-adjacent object.

Projection algorithm

Do not smooth raw `(x, y)` positions after clamping them to an edge. Doing so causes a marker transitioning around a corner to cut through the interior of the screen. Smooth a **direction vector**, then re-intersect the safe rectangle every frame.

For target world position `T` and camera position `C`:

```
d = T - C

right   = camera right unit vector
up      = camera up unit vector
forward = camera forward unit vector

cx = dot(d, right)
cy = dot(d, up)
depth = dot(d, forward)
```

For targets clearly in front of the near plane, keep matrix projection as the authoritative screen placement:

```
Projection p = MarkerProjection.project(relative, modelView, projection);

if (p.isFinite() && depth > nearPlaneEpsilon) {
    direction = new Vec2(p.ndcX(), -p.ndcY());
    front = true;
}
```

For a behind-camera or near-plane target, do **not** rely solely on negative-`w` projection and mirror arithmetic. Use camera-space bearing:

```
Vec2 v = new Vec2(cx, -cy);

if (v.lengthSquared() > 1.0e-8) {
    direction = v.normalize();
} else {
    // Exact/really-near exact rear vector has no stable 2-D projection.
    direction = state.lastNonDegenerateDirection()
        .orElse(new Vec2(0.0, 1.0)); // deterministic bottom-center fallback
}
```

The current implementation already has an exact-behind fallback and safe-rectangle intersection logic, which confirms that this degeneracy exists in the real path; 1.5.4 should move it into explicit stateful handling rather than let sign noise choose an edge.

Determine whether a front-facing target is on screen **before** edge clamping. Use a safe-visible rectangle distinct from the edge indicator rectangle:

```
visibleRect = [margin, width-margin] × [margin, height-margin]
edgeRect    = [edgeInset + radius, width-edgeInset-radius]
              × [edgeInset + radius, height-edgeInset-radius]
```

For edge placement, intersect a ray from screen center with `edgeRect`:

```
static Vec2 intersectRect(Vec2 direction, Rect r, Vec2 center) {
    double dx = direction.x();
    double dy = direction.y();

    double tx = Math.abs(dx) < EPS
            ? Double.POSITIVE_INFINITY
            : halfWidth(r) / Math.abs(dx);

    double ty = Math.abs(dy) < EPS
            ? Double.POSITIVE_INFINITY
            : halfHeight(r) / Math.abs(dy);

    double t = Math.min(tx, ty);
    return center.add(direction.scale(t));
}
```

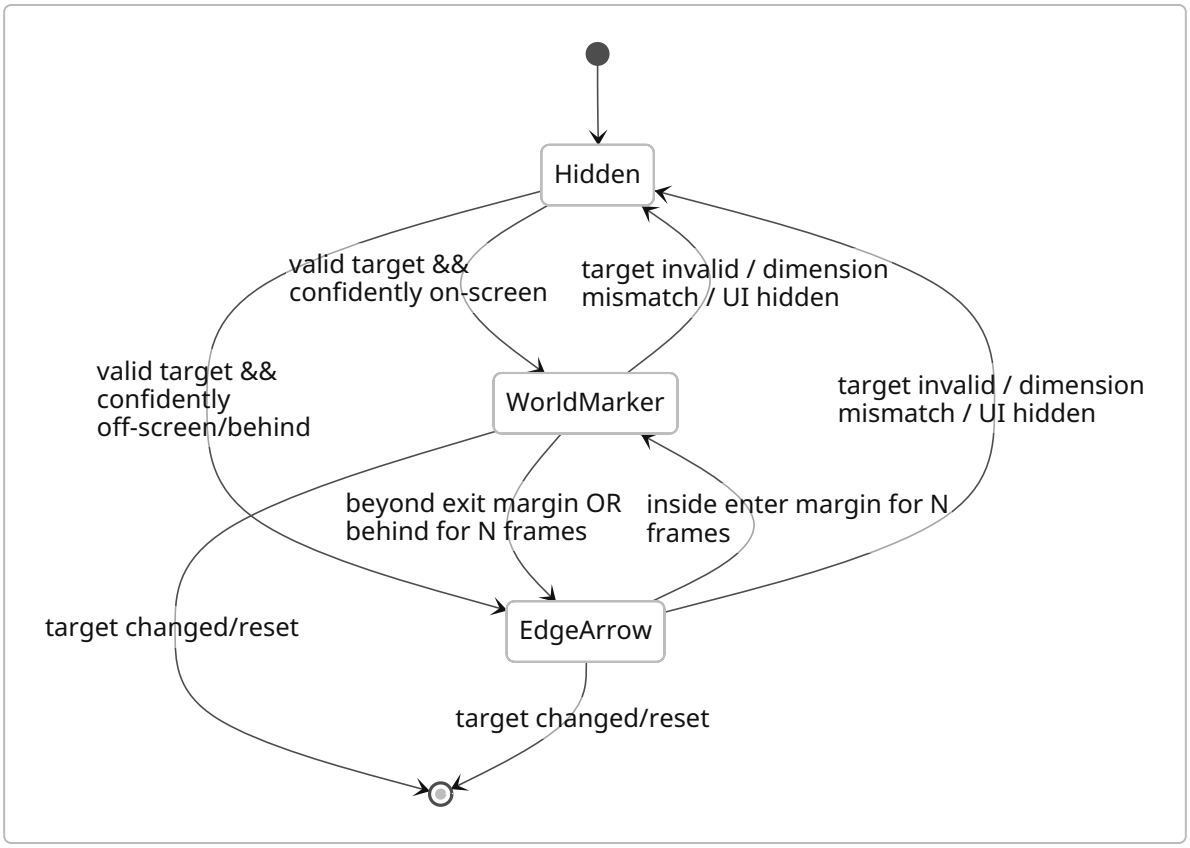
Postcondition:

```
assert Double.isFinite(out.x());
assert Double.isFinite(out.y());
assert edgeRect.containsInclusive(out, 0.5);
```

If any matrix/vector input is non-finite, **do not draw that frame**. Never turn NaN into a casted integer coordinate.

Hysteresis state machine

A target sitting on the exact frustum boundary should not alternate every frame between the world glyph and edge arrow. Introduce separate entry and exit thresholds.



Recommended defaults:

```
edgeEnterHysteresisPx = 4
edgeExitHysteresisPx  = 2
edgeTransitionFrames  = 2
```

Interpretation:

- An edge arrow already visible should not disappear until the projected target is at least 4 GUI pixels **inside** the visible rectangle for two consecutive frames.
- A world marker should not switch to edge mode for a one-frame 1-pixel excursion; it switches when at least 2 pixels outside for two frames.
- `behind=true` can enter edge mode immediately except around the near-plane epsilon, where the two-frame rule applies.
- A dimension mismatch or invalid target bypasses hysteresis and hides immediately.

Reduced-motion mode may disable interpolation, but keeping the spatial dead-zone/hysteresis is permitted because it prevents flicker rather than creating decorative motion.

Smoothing

Use frame-rate-independent exponential smoothing:

```
double alpha(double dtSeconds, double tauSeconds) {
    double dt = Mth.clamp(dtSeconds, 0.0, 0.100);
    return 1.0 - Math.exp(-dt / tauSeconds);
}
```

Recommended `tau = 0.080s`.

Filter the direction:

```
Vec2 filtered = old.scale(1.0 - a).add(raw.scale(a));

if (filtered.lengthSquared() < EPS) {
    filtered = raw;
}

filtered = filtered.normalize();
Vec2 edgePosition = intersectRect(filtered, safeRect, center);
```

This gives approximately a 240 ms 95%-settling window for an 80 ms time constant while behaving similarly at 30, 60, 144 or uncapped FPS.

For displayed orientation, derive the arrow angle from the **same filtered vector**:

```
double angle = Math.atan2(filtered.y(), filtered.x());
```

Do not independently smooth `x`, `y`, and angle; that can make the icon point in a different direction from its edge location.

Discontinuity reset rules

Smoothing state **MUST snap/reset** when any of these occurs:

Discontinuity	Reset behavior
Quest/guidance target identity changes	Full reset
Player dimension changes	Full reset and hide old frame
Target dimension changes	Full reset
Window size or GUI scale changes	Recompute geometry; reset filtered vector
Frame gap > 250 ms	Snap to raw next result

Discontinuity	Reset behavior
Camera teleport > configurable threshold, default 8 blocks	Snap
Target jumps > 16 blocks without continuous entity interpolation	Snap
Leaving world / disconnect	Clear
F1/GUI hidden path stops renderer	Existing frame stamp must prevent stale redraw
Invalid matrix/non-finite projection	Hide frame; preserve last direction only briefly for recovery

The current frame-stamp stale rejection should remain an invariant and retain its unit test.

Target anchor reliability

For an entity target:

```
position = lerp(partialTick, entity.oldPos, entity.pos)
anchorY = lerp(oldY, currentY) + entityBoundingHeight * markerAnchorFraction
```

Keep the existing live-entity resolution/fallback architecture. Do not resolve an entity twice per frame from unrelated paths. The renderer should prepare one immutable `ResolvedGuidanceTarget` and pass it to both world-marker and edge-indicator logic.

Suggested record:

```
public record ResolvedGuidanceTarget(
        UUID logicalTargetId,
        ResourceKey<Level> dimension,
        Vec3 worldAnchor,
        GuidanceKind kind,
        double distance,
        boolean liveEntity,
        long resolutionEpoch
) {}
```

On-screen occlusion policy

Do not conflate “off-screen” and “behind a wall.”

Default:

```
OFFSCREEN_ONLY
```

This preserves the current concept: the edge arrow appears when the target is outside the view or behind the camera; ordinary on-screen terrain occlusion remains controlled by `questMarkerOcclusion`.

Optional P2 mode:

```
OFFSCREEN_OR_OCCLUDED
```

In this mode, if the world marker is on-screen but configured `HIDDEN` due to terrain, an edge/guidance indicator may appear. This **MUST NOT** perform a block raycast every rendered frame.

Recommended sampler:

```
sample interval:      50 ms minimum
camera move trigger:  > 0.25 blocks
camera rotate trigger: > 2 degrees
target move trigger:  > 0.5 blocks
occluded -> visible debounce: 100 ms
visible -> occluded debounce: 100 ms
maximum raycasts:     20/sec for the one tracked target
```

The raycast should use vanilla world clipping from camera eye to target anchor, treating a hit substantially before the target as occluded. Do not attempt a GPU depth-buffer readback; it creates synchronization/performance complexity for little benefit.

HUD collision and accessibility

The safe rectangle must account for icon dimensions, not merely a point inset. The existing indicator is described in repository material as an 18 px edge indicator with distance/angle behavior. For 1.5.4:

```
effectiveInsetX = configuredInset + iconHalfWidth
effectiveInsetY = configuredInset + iconHalfHeight
```

Place distance text **inward** from the edge:

```
left edge   -> text right of icon
right edge  -> text left of icon
top edge    -> text below icon
bottom edge -> text above icon
```

At corners choose the axis with more available label width. Clamp the full text bounding box inside the GUI screen.

Retain:

- high-contrast treatment;
- reduced-motion setting;
- localizable distance formatting;
- no marker when HUD is intentionally hidden;
- no compulsory animation/pulsing.

Performance budget

This subsystem tracks only the active guidance target, so there is no justification for substantial frame cost.

Normative budget on a typical client:

Path	Budget
Projection + state + smoothing, median	≤ `0.05 ms/frame`
Projection + state + smoothing, p99	≤ `0.15 ms/frame`
Heap allocations after warm-up	`0` per frame in marker math
Default occlusion raycasts	`0`
Optional occlusion mode	≤ `20/sec`, one target
Network packets added	`0`

Use mutable scratch vectors or primitive records carefully enough that JIT escape analysis is not the only thing preventing a high-FPS allocation stream.

Projection pseudocode

```
EdgeOutput update(
        ResolvedGuidanceTarget target,
        CameraSnapshot camera,
        Matrix4f modelView,
        Matrix4f projection,
        ScreenMetrics screen,
        double dt
) {
    if (!target.dimension().equals(camera.dimension())) {
        state.clear();
        return EdgeOutput.hidden();
    }
```

```
    if (state.discontinuity(target, camera, screen, dt)) {
        state.resetTemporalFilter();
    }

    CameraSpace c = camera.toCameraSpace(target.worldAnchor());

    Vec2 rawDirection;
    boolean confidentlyOnScreen = false;

    if (c.depth() > NEAR_EPSILON) {
        Projection p = MarkerProjection.project(c.relative(), modelView,
projection);

        if (!p.isFinite()) {
            return EdgeOutput.hidden();
        }

        confidentlyOnScreen =
            screen.visibleRect().containsNdcWithHysteresis(p.ndcX(), p.ndcY(),
state.mode());

        rawDirection = Vec2.of(p.ndcX(), -p.ndcY()).normalizedOrNull();
    } else {
        rawDirection = Vec2.of(c.right(), -c.up()).normalizedOrNull();
    }

    if (rawDirection == null) {
        rawDirection = state.lastNonDegenerateDirection()
            .orElse(Vec2.DOWN);
    }

    EdgeMode mode = state.advanceMode(confidentlyOnScreen, c.depth());

    if (mode == EdgeMode.WORLD) {
        state.remember(rawDirection);
        return EdgeOutput.worldMarker();
    }

    Vec2 direction = settings.reducedMotion()
        ? rawDirection
        : smoother.update(rawDirection, dt);

    Vec2 position = screen.edgeRect().intersectFromCenter(direction);
    state.remember(direction);

    return new EdgeOutput(
        true,
```

```
        position.x(),
        position.y(),
        Math.atan2(direction.y(), direction.x()),
        target.distance()
    );
}
```

Bountiful integration, mappings, and UX

Bountiful's documented model is particularly well suited to a soft compatibility layer. Bounty Boards offer and accept Bounties; their available content depends on Decrees, and board reputation affects rarity and objective discounts. [12] Bounty generation first selects rewards, calculates their worth from quantity × `unitWorth`, and then chooses objectives to match the resulting value, with board reputation effectively reducing the objective burden. [13]

That means MCA: Quests should **not** import Bountiful content into its own quest economy and pretend that MCA difficulty is the same thing as Bountiful worth/reputation. The robust integration is bidirectional but layered:

```
Layer A — coexistence
  No API calls; both mods run normally.

Layer B — content integration
  MCA: Quests contributes Bountiful pool/decree data.
  Bountiful owns generation, timers, worth, reputation and turn-in.

Layer C — UX integration
  MCA quests can guide a player to / interact with a bounty board.

Layer D — completion tracking
  Optional guarded shim emits a semantic event after successful
  Bountiful bounty cash-in.

Layer E — future public API
  Prefer an upstream Bountiful event if one becomes available;
  bridge implementation changes without changing MCA objective data.
```

Bountiful's `1.20.1` source confirms the exact content IDs:

```
bountiful:bounty
bountiful:decree
bountiful:bountyboard   # block and block item
```

and defines the board block entity under its own internal `board-be` identifier.

Its source also has `BountyData` containing objective/reward lists and `tryCashIn`, which first rejects expired bounties, then finishes objectives, grants rewards and consumes the bounty on success. Treat this as an **implementation detail used to understand semantics**, not a public API contract.

MCA-side public compatibility API

```
public interface BountifulBridge extends CompatProvider {

    enum Capability {
        DATA_PACK,
        BOARD_REGISTRY,
        CASH_IN_HOOK,
        READ_RARITY,
        READ_OBJECTIVES
    }

    void addCompletionListener(BountifulCompletionListener listener);

    Optional<BountySnapshot> inspect(ItemStack stack);
}

@FunctionalInterface
public interface BountifulCompletionListener {
    void onCompleted(BountyCompletion completion);
}

public record BountyCompletion(
        UUID playerId,
        long serverGameTime,
        String rarity,              // "UNKNOWN" if unavailable
        int objectiveCount,
        @Nullable ResourceLocation sourceDecree,
        String dedupeKey
) {}

public record BountySnapshot(
        String rarity,
        int objectiveCount,
        int rewardCount
) {}
```

There must be no `io.ejekta.*` type in these signatures.

Implementations:

```
NoopBountifulBridge
DataOnlyBountifulBridge
HookedBountifulBridge
```

Selection:

```
if (!modPresent("bountiful")) {
    return new NoopBountifulBridge();
}

if (!canBindCashInHook()) {
    return new DataOnlyBountifulBridge();
}

return new HookedBountifulBridge();
```

Cash-in hook

Because no documented public completion event was found, the 1.5.4 bridge may use a guarded compatibility mixin/injection targeted at the successful return of Bountiful's `BountyData.tryCashIn`.

Requirements:

- The mixin/injection is loaded **only** when Bountiful is present.
- A mixin config plugin or equivalent must check target class/method availability before application.
- Failure to bind logs one concise warning and leaves `DATA_PACK` /board integration alive.
- Never cancel, modify or replace `tryCashIn`.
- Emit only when original method returned `true`.
- Do not grant or withhold Bountiful rewards.
- Do not mutate `BountyData`.
- Put the implementation behind the bridge so a future official event can replace the mixin without changing quest JSON.

Conceptual injection:

```
// Pseudocode: actual mapped descriptor is loader/version-specific.
@Inject(method = "tryCashIn", at = @At("RETURN"))
private void mcaquests$afterTryCashIn(
        Player player,
        ItemStack bountyStack,
        CallbackInfoReturnable<Boolean> cir
) {
    if (!Boolean.TRUE.equals(cir.getReturnValue())) {
        return;
```

```
    }

    BountifulCompatEvents.emitSuccessfulCashIn(
        player,
        SafeBountySnapshotReader.readOrUnknown(bountyStack)
    );
}
```

The hook itself is high risk because it touches an internal Kotlin class. Consequently, `CASH_IN_HOOK` must be independently probeable from `DATA_PACK`.

MCA objective

Add:

```
{
  "type": "mcaquests:bountiful_bounties",
  "count": 3
}
```

Optional capability-dependent filtering:

```
{
  "type": "mcaquests:bountiful_bounties",
  "count": 1,
  "min_rarity": "RARE"
}
```

Validation rule:

```
count-only objective:
  requires CASH_IN_HOOK

min_rarity:
  requires CASH_IN_HOOK + READ_RARITY
```

Never silently ignore `min_rarity` if `READ_RARITY` is unavailable. Suspend/withhold the quest and report the missing capability.

Progress event:

```
void onBountyCompleted(ServerPlayer player, BountyCompletion event) {
    for (ActiveQuest q : questState.activeFor(player)) {
        q.objectivesOfType(BountifulBountiesObjective.class).forEach(obj -> {
            if (obj.matches(event)) {
                obj.increment(1);
            }
        });
    }
    questState.flushIfDirty(player);
}
```

`tryCashIn` should already produce only one success for the consumed bounty, but the bridge should still apply a short dedupe cache:

```
key = player UUID + dedupeKey + serverGameTime
TTL = 2 server ticks
```

This protects against duplicated compatibility callbacks without creating long-lived state.

Board interaction objective

Add a generic block interaction objective, useful beyond Bountiful:

```
{
  "type": "mcaquests:interact_block",
  "block": "bountiful:bountyboard",
  "count": 1
}
```

It should progress from a server-side successful block-use event and never cancel the original interaction.

This supports an introductory MCA quest:

> “The village has started posting work publicly. Check the bounty board and see what people need.”

The quest objective completes when the player actually interacts with the board; the Bountiful screen still opens and remains Bountiful's UX.

Data mapping

Bountiful's current customization documentation describes `item`, `item_tag`, `entity`, and `criteria` entries, with `amount`, `unitWorth`, rarity, optional conditions, forbids and reputation requirements; the `1.20.1` source also registers a `command` logic type. [14]

Bountiful representation	Closest MCA representation	Conversion policy
`item` objective	`item_delivery` or `gather`	Use `item_delivery` when mirroring turn-in semantics; use `gather` only for possession-style content.
`item_tag` objective	Tag-based item delivery/gather	Safe if MCA tag objective supports same registry semantics.
`entity` objective	`kill_entity`	Safe by exact registry ID.
`criteria`	Advancement/ external criterion	**Do not auto-convert generally.** Bountiful criteria are event-like and lack advancement-style memory. [8]
`command`	Command reward/ action	Never auto-import; MCA command execution remains separately permission/config gated.
`amount.min/ max`	Frozen objective count	Roll once server-side on quest generation/acceptance, mirroring MCA's freeze-at-accept principle.
`unitWorth`	No exact equivalent	Preserve only as Bountiful-side economy data.
`rarity`	Integration metadata	Keep Bountiful rarity string; do not force it into MCA difficulty.
`repRequired`	Bountiful board gating	Do not map automatically to MCA village reputation; the two reputation systems have different scopes. Bountiful reputation belongs to individual boards. [15]
Pool	MCA category/source metadata	One-to-many relation; not a quest ID.
Decree	Quest source/theme metadata	Do not equate with MCA profession.

Bountiful compatibility data

Bountiful's data loading is filename-oriented and merges resources found under its pool/decree folders; its source groups loaded resources by file name and derives the resulting data ID from that file name. This creates an important compatibility rule:

> MCA: Quests compatibility pool/decree filenames **MUST be globally distinctive**, not merely placed under a distinctive namespace.

Recommended:

```
data/mcaquests/bounty_pools/mcaquests_iafce_hunts.json
data/mcaquests/bounty_pools/mcaquests_iafce_materials.json
data/mcaquests/bounty_decrees/mcaquests_dragons.json
```

Before implementation, use the targeted Bountiful branch/runtime to verify its exact resource-pack search folder path because Bountiful has changed data layouts across versions. Its official docs explicitly expose version-specific customization pages, and the current source loader searches resources by the configured folder name across the resource manager. [16]

Example CE hunt pool:

```
{
  "content": {
    "mcaquests_iafce_fire_dragon": {
      "type": "entity",
      "content": "iceandfire:fire_dragon",
      "amount": { "min": 1, "max": 1 },
      "unitWorth": 12000,
      "rarity": "RARE"
    },
    "mcaquests_iafce_ice_dragon": {
      "type": "entity",
      "content": "iceandfire:ice_dragon",
      "amount": { "min": 1, "max": 1 },
      "unitWorth": 12000,
      "rarity": "RARE"
    },
    "mcaquests_iafce_lightning_dragon": {
      "type": "entity",
      "content": "iceandfire:lightning_dragon",
      "amount": { "min": 1, "max": 1 },
      "unitWorth": 14000,
      "rarity": "RARE"
    }
  }
}
```

The numeric values above are **design defaults requiring play-balance validation**, not upstream values. Bountiful's generator uses `unitWorth` to match objective burden to rewards, so the release process must validate them against actual packs rather than assume an absolute universal price scale. [13]

A better balance method is percentile-based:

```
1. Load Bountiful's effective pool after merging.
2. Compute typical unitWorth distribution for comparable hunt objectives.
3. Put Hydra / dragon / sea-serpent objectives in the intended percentile.
4. Run Bountiful's validation/sampling tools or equivalent generation tests.
5. Adjust until generated bounties do not regularly require absurd objective
stacks.
```

Bountiful documentation provides testing/sample tooling for detecting mismatched objective/reward economies in versions that expose those commands. [17]

Conditional pack activation

A pack referencing `iceandfire:*` should not be blindly enabled whenever Bountiful is installed.

Implement:

```java
public interface ConditionalCompatPack {
    ResourceLocation id();
    boolean shouldEnable(CompatRegistry capabilities);
    void register(PlatformPackRegistrar registrar);
}
```

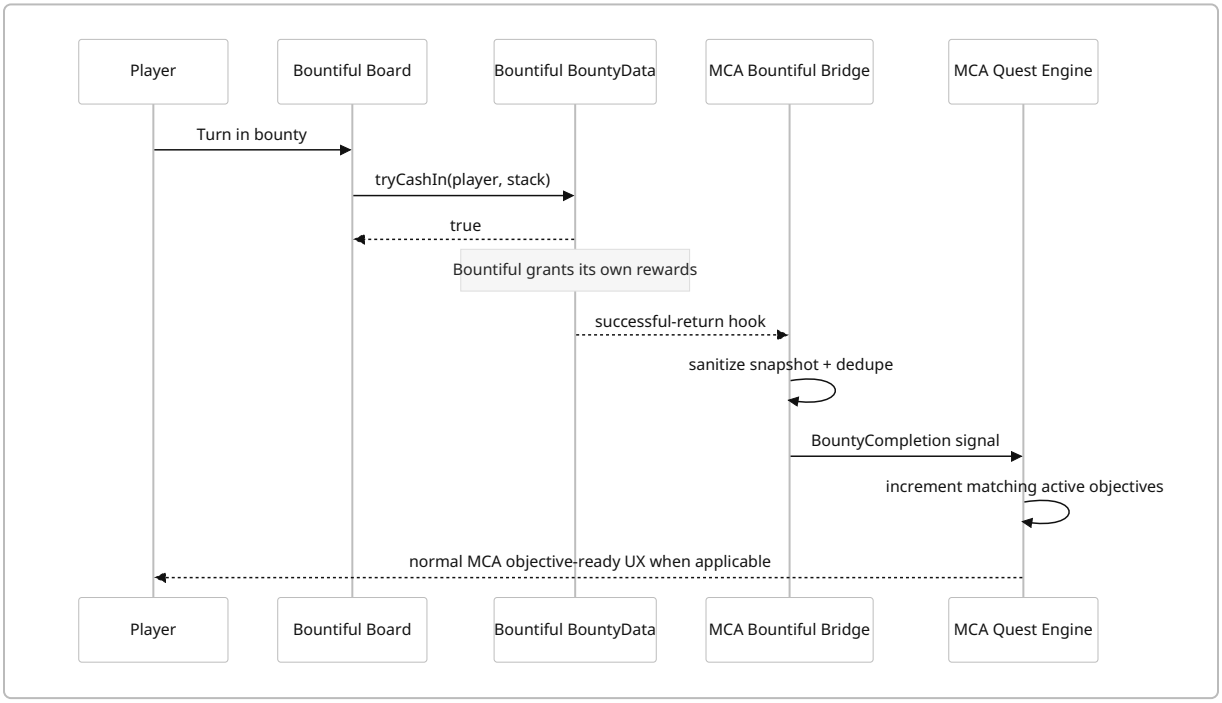
Rules:

```
mcaquests_bountiful_core:
    requires bountiful board registry

mcaquests_bountiful_iafce:
    requires bountiful + iceandfire.core
    content entries additionally filtered/probed where possible

mcaquests_iafce_quests:
    requires iceandfire.core
```

Where the loader cannot conditionally expose embedded datapacks cleanly, use pre-load resource filtering or ship content that Bountiful itself can safely filter only **after a test proves missing registry IDs are non-fatal**. Do not rely on undocumented graceful failure.

Bountiful UX flows

Flow	Player experience	Technical behavior
Discovery	MCA villager asks player to inspect the bounty board	`interact_block bountiful:bountyboard`; no Bountiful internals
Contractor	“Complete two posted bounties, then return”	`bountiful_bounties count=2`; completion shim increments
Specialist	“Complete a rare contract”	Requires `READ_RARITY`; otherwise quest is not offered
Dragon hunter	Bountiful board generates CE dragon/material contracts	Conditional Bountiful pool pack; Bountiful owns generation/turn-in
Missing Bountiful	Existing contractor quest says integration unavailable and remains abandonable	`SUSPENDED_COMPAT`, progress retained
Bountiful restored	Quest resumes at previous count	Reprobe capabilities during server start/reload



Configuration, data model, migration, localization, and refinements

The existing MCA: Quests repository already separates common gameplay configuration from client visual configuration and keeps quest content in datapacks. [3] Preserve that separation.

Common configuration additions

Recommended TOML-facing schema:

```toml
[compat]
    diagnostics = true

[compat.iceandfire]
    enabled = true
    flavor = "AUTO" # AUTO, ORIGINAL, COMMUNITY_EDITION, REGISTRY_ONLY, OFF
    enableBuiltinContent = true
    strictRegistryValidation = true
    suspendUnavailableObjectives = true

[compat.bountiful]
    enabled = true
    mode = "AUTO" # AUTO, DATA_ONLY, HOOKED, OFF
    enableBuiltinContent = true
    completionTracking = true
    suspendOnHookLoss = true
    enableIceAndFirePools = true

[compat.validation]
    failDatapackReloadOnInvalidOptionalIds = false
    logMissingOptionalContent = true
```

Semantics:

```
AUTO:
    choose highest verified capability mode

REGISTRY_ONLY:
    I&F may be used by IDs but no optional behavioral adapters

DATA_ONLY:
    Bountiful pool/decree integration, no cash-in hook

HOOKED:
    require successful cash-in capability; otherwise degrade with warning,
    unless explicitly configured as strict

OFF:
    no integration content or hooks
```

Avoid a configuration that makes optional-mod absence a server-start fatal error by default.

Client marker configuration

Keep old `questMarkerEdgeIndicator` readable for backward compatibility.

Add:

```
[marker.edge]
    mode = "OFFSCREEN_ONLY" # DISABLED, OFFSCREEN_ONLY, OFFSCREEN_OR_OCCLUDED
    inset = 18
    smoothingMs = 80
    enterHysteresisPx = 4
    exitHysteresisPx = 2
    transitionFrames = 2
    showDistance = true
    occlusionSampleMs = 50
```

Migration rule:

```
if (!newConfigHasExplicit("marker.edge.mode")) {
    mode = oldQuestMarkerEdgeIndicator
        ? OFFSCREEN_ONLY
        : DISABLED;
}
```

Do not remove or rename the legacy boolean in 1.5.4. Document it as deprecated/internal compatibility input and consider removal only in a future breaking-config release.

Clamp user settings:

```
inset:               0..128 GUI px
smoothingMs:         0..500
enterHysteresisPx:   0..32
exitHysteresisPx:    0..32
transitionFrames:    1..10
occlusionSampleMs:   25..1000
```

Persistent compatibility state

Do not add compatibility state to every quest unless needed.

Suggested additive saved structure:

```
{
  "mcaquestsDataVersion": 154,
  "activeQuests": [ "...existing data..." ],
  "compat": {
    "schema": 1,
    "suspensions": {
      "quest-instance-uuid": {
        "provider": "iceandfire",
        "capability": "iceandfire.myrmex",
        "previousState": "ACTIVE"
      }
    }
  }
}
```

An alternative—and preferable design if the existing active quest schema already supports suspension metadata—is to reuse that generic field and add only:

```
{
  "state": "SUSPENDED_COMPAT",
  "suspension": {
    "provider": "bountiful",
    "capability": "CASH_IN_HOOK"
  }
}
```

Do not serialize detected optional-mod versions as authoritative data. They are environment observations and should be re-probed each load.

Migration algorithm

```
Load old data
   |
   +-- New fields absent?
   |      -> populate runtime defaults only
   |
   +-- Active objective references existing registry ID?
   |      -> leave untouched
   |
   +-- Active objective declares compat capability now missing?
   |      -> suspend
   |
   +-- Legacy I&F objective references Myrmex ID and registry ID missing?
   |      -> suspend only if quest can be confidently classified as optional
```

```
   |
   +-- Bountiful objective from pre-release/dev schema?
          -> normalize aliases, preserve count/progress
```

Critical rule:

> **Never rewrite an existing I&F registry ID into a "CE equivalent" unless a documented one-to-one semantic mapping exists.**

Myrmex does not have a CE equivalent; it is absent. [10] Substitution would silently change the user's quest.

Quest-definition backward compatibility

All existing files at MCA: Quests' datapack location must keep working. The repository documents quests as data under `data/<namespace>/mcaquests/quests/**.json`, with validation/reload commands. [3] New functionality is additive:

```
{
  "conditions": [
    {
      "type": "mcaquests:compat_capability",
      "provider": "iceandfire",
      "capability": "iceandfire.dragon_seekers"
    }
  ]
}
```

No required top-level schema-version bump is needed merely to add a registered condition/objective type.

General refinements recommended for 1.5.4

The compatibility work exposes reusable features worth implementing rather than leaving integration-specific hacks:

Refinement	Specification
`compat_capability` condition	Generic offer gating for any optional integration
`use_item` objective	Server-side exact item-ID use tracking
`interact_block` objective	Server-side exact block/tag interaction tracking
Generic suspension reason	`SUSPENDED_COMPAT` with provider/capability

Refinement	Specification
Registry validator	Validates optional entity/item/block/structure IDs after registries are ready
Compatibility diagnostics	Human-readable status, probe, missing IDs and capabilities
Conditional pack registry	Loader abstraction for optional embedded data
Build linkage tripwire	Fails build for prohibited direct optional-mod type descriptors
Dev overlay	Optional debug arrow showing raw vs filtered direction and state, disabled in production by default

Diagnostics commands

Add:

```
/mcaquests compat status
/mcaquests compat iceandfire status
/mcaquests compat iceandfire probe
/mcaquests compat bountiful status
/mcaquests compat bountiful probe
/mcaquests debug marker
```

Example output:

```
Ice & Fire
  flavor: COMMUNITY_EDITION
  class probe: com.iafenvoy.iceandfire.IceAndFire
  core entities: OK
  lightning dragon: OK
  myrmex: MISSING (expected for CE)
  dragon seekers: OK
  netherite dragon armor: OK
  mode: REGISTRY_ONLY

Bountiful
  mod: present
  bountyboard: OK
  data pack: OK
  cash-in hook: OK
  rarity reader: UNKNOWN
  effective mode: HOOKED
```

Diagnostics must distinguish an **expected absent capability** from an error. “Myrmex missing under CE” is informational, not a warning.

Localization

Every player-visible string introduced by this release must be a translation key, including compatibility suspension reasons, config-screen labels if present, quest objective text, diagnostics intended for players, marker accessibility labels, Bountiful objective descriptions and CE content. MCA: Quests already emphasizes fully localizable content and automated locale parity checks; preserve that test discipline. [3]

Suggested keys:

```
{
  "mcaquests.compat.iceandfire.name": "Ice & Fire",
  "mcaquests.compat.iceandfire.ce": "Ice & Fire: Community Edition",
  "mcaquests.compat.bountiful.name": "Bountiful",

  "mcaquests.quest.suspended.compat":
    "Quest paused: required content from %s is unavailable.",

  "mcaquests.objective.bountiful_bounties.one":
    "Complete %s Bountiful bounty",
  "mcaquests.objective.bountiful_bounties.many":
    "Complete %s Bountiful bounties",

  "mcaquests.objective.interact_block":
    "Use %s",
  "mcaquests.objective.use_item":
    "Use %s",

  "mcaquests.marker.distance.blocks":
    "%s blocks"
}
```

Do not concatenate grammar-sensitive fragments such as `"Complete " + count + " bounties"`. Keep whole sentences translatable.

For languages already promised/supported by the repository, the release gate should require key-set parity. A missing community translation may temporarily reuse English text only if the project's established localization policy allows it; missing keys themselves must fail the parity test.

Testing and QA specification

Testing must be split between pure algorithm tests, compatibility contract tests, game integration tests, migration fixtures, and manual UX/performance QA. A compatibility release is not adequately tested by “the game starts with all mods installed.”

Unit tests — marker mathematics

Create or expand:

```
MarkerProjectionTest
EdgeSafeRectTest
EdgeIndicatorStateTest
EdgeIndicatorSmootherTest
MarkerFrameStateTest
```

Mandatory cases:

Test	Expected invariant
Target centered in front	World marker, no edge arrow
Left/right/up/down beyond frustum	Correct screen edge
Four diagonal quadrants	Correct edge/corner intersection
Directly behind	Deterministic stable fallback
Behind-left/right	Corresponding useful turn direction
Near-plane `depth = ±epsilon`	No NaN; no rapid edge flip
`w → 0` matrix projection	Safe fallback/hide
NaN/Infinity input	Hidden, never rendered
320×240-like small GUI	Full icon and label remain visible
Ultrawide aspect	Correct edge intersection
GUI scale change	Reset then correct position
FOV change	Correct new projection, no stale position
Tiny camera noise at boundary	Hysteresis prevents flicker
30 vs 144 FPS trajectory	Approximately identical temporal response
Target change while smoothing	Immediate reset, no travel from old target
Dimension change	Old frame cannot persist
GUI/world renderer skipped	Existing stale-frame test remains passing

Add property tests over randomized finite direction vectors:

```
forAllFiniteDirections(d -> {
    Vec2 p = rect.intersectFromCenter(d);
    assertTrue(Double.isFinite(p.x()));
    assertTrue(Double.isFinite(p.y()));
```

```
    assertTrue(rect.containsInclusive(p, 1e-6));
    assertTrue(rect.isOnBoundary(p, 1e-6));
});
```

A second property test should prove that smoothing output remains normalized/finite and edge intersection remains valid for random `dt ∈ [0, 0.1]`.

Unit tests — Ice & Fire compatibility

Use a fake registry/capability probe rather than loading I&F classes into JUnit.

```
no class + no IDs                -> NONE
CE class + core IDs              -> COMMUNITY_EDITION
original class + core IDs        -> ORIGINAL
both classes                     -> AMBIGUOUS / registry-only
CE + seekers                     -> dragon_seekers true
CE without seekers               -> false, no crash
CE without Myrmex                -> myrmex false
future CE with Myrmex IDs        -> myrmex true
missing lightning dragon         -> only lightning capability false
technical projectile IDs         -> never enter kill allowlist
```

Manifest test:

```
every declared quest-safe entity ID:
    syntactically valid ResourceLocation

every excluded technical ID:
    cannot also appear in quest-safe list

CE 1.20.1 expected manifest:
    matches checked-in snapshot from upstream registry audit
```

Add a source/build tripwire that scans compiled class constant pools or byte strings. Outside the isolated optional-compat target strings, fail if production classes contain:

```
com/iafenvoy/iceandfire/
com/github/alexthe666/iceandfire/
io/ejekta/bountiful/
```

MCA: Quests already documents this kind of build-time direct-link prevention for fragile integrations, making it an established project pattern rather than a novel constraint. [3]

Unit tests — Bountiful

```
BountifulBridgeSelectionTest
BountifulCompletionDedupeTest
BountifulObjectiveTest
BountifulDataMappingTest
ConditionalCompatPackTest
```

Cases:

```
Bountiful absent -> Noop bridge
Bountiful present / hook unavailable -> DATA_ONLY
Hook available -> HOOKED
cash-in false -> no signal
cash-in true -> one signal
duplicate callback same tick -> one progress increment
two distinct bounties -> two increments
count objective + UNKNOWN rarity -> valid
min_rarity objective + UNKNOWN rarity -> suspended/unsupported
Bountiful removed with active objective -> suspended, progress retained
Bountiful restored -> resumes
```

Integration matrix

The concrete 1.20.1 Forge baseline should cover at least:

MCA: Quests	I&F	Bountiful	Expected
✓	none	none	Baseline unchanged
✓	original	none	Original content remains functional
✓	CE	none	CE detected, Myrmex absent/gated
✓	none	✓	Data/board/completion integration
✓	original	✓	All coexist
✓	CE	✓	CE pools + Bountiful + MCA objectives
✓	CE	removed after save	CE-dependent progress suspends
✓	none	Bountiful removed after save	Bountiful objective suspends

MCA: Quests	I&F	Bountiful	Expected
✓	CE	✓ then version mismatch in cash-in hook	Data-only fallback, server stays alive

Also test with existing map-marker integrations because changing marker state must not regress JourneyMap/Xaero behavior; MCA: Quests already treats map compatibility separately from its world/HUD marker path. [3]

I&F CE manual acceptance cases

Case	Procedure	Pass
Three dragons	Kill fire, ice, lightning dragon for exact-ID objectives	Correct target increments once
Hydra/Dread roster	Generate representative hunt quests	Only living quest-safe IDs appear
Myrmex	Run CE alone and inspect offers	No Myrmex quest offered
Existing Myrmex active save	Load fixture containing active Myrmex objective	Quest suspended, progress preserved, no crash
Seekers	Obtain/use normal, epic, legendary	`use_item` tracks exact configured tier
Godly seeker	Normal survival offer generation	Never required by default
Netherite dragon armor	Acquire four armor parts	Exact-ID gather/craft objectives work
Scales	Obtain scales using CE brush route	Existing gather objective sees resulting inventory normally
Dragon blood	Craft through CE-supported path	Result acquisition works without I&F internal API
Registry mismatch	Remove one expected item using test fixture/mock	Only dependent content disabled
Dedicated server	Start with CE present	No client-only I&F/MCA marker classloading failure

Bountiful manual acceptance cases

Bountiful's normal board flow is to pick up and turn in bounties, with Decrees selecting themed pools and board reputation affecting the generated offers. [12] Tests must preserve that flow rather than bypass it.

Case	Pass
Interact with `bountiful:bountyboard` during MCA introduction quest	MCA objective completes; Bountiful UI still opens
Complete valid bounty	MCA counter increments once after Bountiful succeeds
Attempt incomplete bounty	No MCA progress
Attempt expired bounty	No MCA progress; Bountiful retains its own failure behavior
Two active MCA quests track bounties	Both progress if their filters match
Rare-filter objective	Counts only verified qualifying rarity
CE hunt pool	Board can generate configured CE entity contracts
Friend/player copies	MCA doesn't interfere with Bountiful's per-player board inventory behavior. Bountiful documents that different players can take their own copies of the same displayed bounty. [18]
Remove MCA: Quests	Bountiful remains ordinary Bountiful; no Bountiful data corruption
Remove Bountiful	MCA world loads; Bountiful-dependent objectives suspend
`/reload`	Data pools and MCA definitions rebuild without duplicate registrations

Marker manual acceptance matrix

Test each at GUI scales `1`, `2`, `3/auto`, at 30 FPS cap, 60 FPS, and ≥144 FPS where available:

```
target dead ahead
target just beyond each screen edge
target behind at 180°
target behind-left and behind-right
target far above/below
slow pan across screen boundary
rapid 360° camera spin
small mouse jitter at boundary
player teleport
target teleport
entity entering/leaving render distance
entity death/despawn
dimension portal
F1 toggle
window resize
```

```
fullscreen toggle
FOV minimum/default/high
first-person / third-person
spectator where supported
pause/unpause
wall between camera and target
occlusion mode FULL / DIM_OUTLINE / HIDDEN
high-contrast
reduced-motion
```

Pass criteria:

```
No indicator flicker faster than the configured state hysteresis.
No visible one-frame jump to the opposite edge at exact rear alignment.
No arrow outside safe rectangle.
No distance label clipped by screen boundary.
No stale arrow after target/world disappearance.
No direction that materially disagrees with shortest useful turn direction.
No smooth transition that crosses the screen interior.
```

Performance QA

Use a client profiler/JFR or equivalent to instrument:

```
MarkerProfiler.begin();
// projection + state
MarkerProfiler.end();
```

Record median and p99 over at least several thousand frames while rotating around a tracked target.

Release thresholds:

```
normal edge path median <= 0.05 ms
normal edge path p99    <= 0.15 ms
no sustained allocation in projection/state hot path
default path raycasts   = 0
optional occlusion      <= configured rate
```

Server-side compatibility listeners must not scan all entities/world blocks per tick. Bountiful progress is event-driven; I&F capability probing occurs at load/reload, not every quest tick.

Migration/save fixtures

Check into `src/test/resources/migration/` sanitized fixtures representing:

```
pre-1.5.4 ordinary active quest
pre-1.5.4 tracked entity objective
pre-1.5.4 I&F dragon quest
active Myrmex quest
completed quest archive
projects/situations unaffected by integrations
old marker client config with edge true
old marker client config with edge false
```

Round-trip requirement:

```
load old fixture
-> migrate in memory
-> save 1.5.4
-> reload 1.5.4
-> assert semantic state equal
```

No test should require downgrading the 1.5.4 save into an older MCA: Quests version unless the project already guarantees downgrade compatibility.

Developer milestones, task breakdown, and definition of done

The milestones below are ordered to reduce rework: compatibility infrastructure precedes mod-specific content, and marker math can proceed in parallel because it does not affect save formats.

Milestone	Deliverables	Effort	Exit gate
Compatibility foundation	`CompatRegistry`, capability condition, suspension reason, diagnostics skeleton, platform hooks	Medium	Unit tests + no optional-mod static references
I&F CE core	Flavor detector, registry manifest, capability probe, Myrmex gating, quest-safe entity list	Medium	Original + CE smoke worlds start and validate
I&F CE content	Seeker/netherite content, generic `use_item`, updated built-ins/datapacks, localization	Medium	CE manual matrix passes
Marker hardening	Direction-based projection, safe rectangle, hysteresis, smoother, reset rules, tests	Medium	Algorithm/property/performance tests pass

Milestone	Deliverables	Effort	Exit gate
Bountiful data layer	Conditional packs, pools/decrees, board interaction objective	Medium	Bountiful loads and generates test content
Bountiful event layer	`BountifulBridge`, guarded cash-in hook, objective, dedupe, diagnostics	High	Successful/failed cash-in semantic tests pass
Migration and polish	Config migration, suspension migration, localization parity, docs	Medium	Old fixtures round-trip
Release QA	Full mod matrix, dedicated server, profiler, validation/build	High	All acceptance criteria pass

Concrete coding tasks

The coding agent should execute the work in approximately this dependency order:

Task	Files/modules	Notes
Define generic `CompatProvider` / `CompatRegistry`	`compat/*`	Must not depend on optional mod classes
Add `compat_capability` quest condition	condition registry + datapack docs	Additive schema
Add `SUSPENDED_COMPAT` semantics	active quest lifecycle/state	Preserve abandonability/ progress
Add compatibility diagnostics commands	command module	Provider-neutral
Implement `IceAndFireCompat` flavor probe	`compat/iceandfire`	Class names only as strings
Implement I&F registry probes	same	Registries are source of truth
Add checked-in CE `1.20.1` manifest	resources	Diagnostic expectations only
Audit existing `iceandfire:` quest references	datapacks	Classify core / Myrmex / original-only
Gate Myrmex content	datapacks/conditions	No substitute target
Add `use_item` objective	generic objectives/events	Needed by seekers and future mods
Add CE seeker/netherite quests	optional datapack	Gate by capabilities
Refactor edge marker around `EdgeIndicatorState`	client marker	Keep `MarkerFrameState` boundary

Task	Files/modules	Notes
Add smoothing/hysteresis	client marker	Direction vector, not clamped position
Add marker property/perf tests	test/client/marker	Zero normal-path raycasts
Add `interact_block` objective	generic objective	Enables board UX
Add `BountifulBridge`	`compat/bountiful`	No Bountiful types in API
Add Bountiful content pack	resources	Unique pool/decree filenames
Add guarded cash-in hook	optional mixin/platform	Never modify Bountiful result/state
Add `bountiful_bounties` objective	objectives	Requires capability
Add Bountiful completion dedupe	compat bridge	Tiny bounded cache
Add config keys + legacy bridge	config	Legacy edge boolean retained
Add localization	`assets/mcaquests/lang/*`	Key parity required
Add migration fixtures	tests/resources	Include unavailable integrations
Run full mod matrix	dev runs/CI	MCA-only must remain first-class
Update docs/changelog	README, CONFIG, DATAPACK, compat docs	Explain degraded modes

Recommended class ownership

Class/module	Responsibility	Explicitly must not do
`CompatRegistry`	Lifecycle/probing/status of optional integrations	Know I&F/Bountiful implementation details
`IceAndFireCompat`	Flavor and capabilities	Import I&F Java classes
`IceAndFireRegistryManifest`	Expected IDs per known target	Decide actual runtime availability without registry probe
`BountifulBridge`	Stable MCA-side semantic contract	Expose Kotlin/Bountiful types
`HookedBountifulBridge`	Translate successful cash-in into MCA event	Mutate rewards, bounty timers or reputation

Class/module	Responsibility	Explicitly must not do
`ConditionalCompatPackRegistrar`	Expose data packs only under valid capability combinations	Perform quest progression
`MarkerProjection`	Pure math	Hold smoothing state
`EdgeIndicatorState`	Hysteresis/reset/last-direction state	Draw GUI
`EdgeIndicatorSmoother`	Temporal filtering	Query world/entities
`QuestMarkerRenderer`	Resolve one target and publish frame state	Duplicate guidance resolution
`QuestMarkerHud`	Draw already-prepared HUD frame	Reconstruct world projection
`MarkerOcclusionSampler`	Optional bounded line-of-sight samples	Run on default path or every frame

Build gates

Add automated gates before a `1.5.4` artifact is considered releasable:

```
./gradlew clean test build
```

plus project-specific validation equivalent to:

```
datapack schema/registry validation
en_us ↔ supported-locale key parity
forbidden optional-package constant-pool scan
saved-data migration fixture suite
dedicated-server classloading smoke test
```

For a coding agent, “compiles successfully” is not a completion condition. The following **definition of done** is normative:

```
[ ] Existing MCA-only environment behaves identically except documented fixes.
[ ] CE identified separately from original I&F.
[ ] No direct I&F implementation linkage.
[ ] No direct Bountiful implementation linkage outside isolated guarded hook
target.
[ ] All optional IDs are registry validated.
[ ] Myrmex quests absent under stock CE.
```

```
[ ] Active unavailable optional objectives suspend/resume.
[ ] CE-exclusive seeker/netherite content validates and works.
[ ] Edge arrow passes near-plane/exact-behind/boundary property tests.
[ ] Edge arrow meets CPU/allocation budget.
[ ] Old edge config is honored.
[ ] Bountiful data integration functions without completion hook.
[ ] Successful Bountiful cash-in emits exactly one MCA completion event.
[ ] Failed/expired/incomplete bounty emits none.
[ ] Removing Bountiful does not break world load.
[ ] Existing save fixtures round-trip without progress loss.
[ ] Localization parity passes.
[ ] Dedicated server starts with every supported optional-mod combination.
[ ] README / CONFIG / DATAPACK / CHANGELOG explain 1.5.4 behavior.
```

Suggested release-note wording

> **Ice & Fire: Community Edition compatibility:** MCA: Quests now distinguishes I&F CE from the original implementation, validates content through Minecraft registries, supports CE-specific Dragon Seekers and Netherite dragon equipment, and safely hides or suspends unavailable Myrmex content.
>
> **Quest marker reliability:** The screen-edge objective indicator has been hardened for targets behind the camera, near the projection plane, at screen boundaries, during teleports/resizes, and at varying frame rates. Direction smoothing and transition hysteresis reduce jitter without changing server quest state.
>
> **Bountiful integration:** Bountiful bounty boards can participate in MCA quest flows and compatibility content. Bountiful continues to own its bounty generation, rewards, reputation and turn-in mechanics; MCA: Quests listens only for successful completion where the compatibility capability is available.

Prioritized primary sources

The implementation should treat source code for the exact targeted branch/tag as authoritative for registry IDs and method shapes, while using maintained project documentation for intended behavior.

Priority	Source	Why it matters
Highest	MCA: Quests GitHub repository	Existing architecture, dependencies, optional-compat philosophy, save/data model and marker implementation. Current README documents the server-authoritative/datapack model, optional bridges and runtime compatibility approach. [3]
Highest	MCA: Quests DATAPACK.md	Existing quest/objective/condition schema; new types must be additive to this contract.

Priority	Source	Why it matters
Highest	MCA: Quests CONFIG.md	Existing marker/config names and defaults; required for backward-compatible configuration migration.
Highest	Existing MCA marker design spec	Establishes the current marker design context; 1.5.4 is a reliability follow-up, not a greenfield HUD.
Highest	Ice & Fire: Community Edition GitHub	Exact CE registry/source truth. Upstream describes CE as a fork with rewrites/new content, no Myrmex, and divergent MC branches. [2]
Highest	I&F CE New Content docs	Seeker behavior/radii, Netherite Dragon Armor and alternate scale acquisition. [11]
High	I&F CE issue demonstrating original-package linkage failure	Concrete evidence for the “no direct original-package linkage” requirement. [9]
Highest	Bountiful GitHub	Exact `1.20.1` Kotlin/source model, registered content, loader bridge and bounty completion semantics. Bountiful itself supports Forge/Fabric lines. [19]
Highest	Bountiful customization docs	Pool entry fields, types, decrees, rarity, worth and criteria caveats. [14]
High	Bountiful generation docs	Defines reward-first worth matching and board-reputation discount behavior; critical to balancing compatibility pools. [13]
High	Bountiful Bounty Boards docs	Canonical board UX, Decree relationship and per-player bounty copies. [18]
High	Bountiful Reputation docs	Confirms that Bountiful reputation is board-attached and modifies rarity/discounts, so it should not be conflated with MCA village reputation. [15]

The central implementation rule across all three goals is the same: **persist MCA-owned semantic state, resolve optional content through stable registry/data boundaries first, isolate unavoidable implementation-specific hooks behind capability probes, and fail by disabling only the dependent feature rather than destabilizing the quest engine.** That strategy is consistent with MCA: Quests' existing compatibility architecture, accommodates I&F CE's deliberately divergent internals, respects Bountiful's own data/economy model, and turns the 1.5.4 marker work into a testable mathematical subsystem instead of another render-path special case. [20]

---

[1] [3] [4] [6] [20] GitHub - otectus/MCAQuests: MCA: Quests is a mod which expands on the MCA: Reborn mod by allowing the player to accept (or decline) quests from MCA villagers. · GitHub
https://github.com/otectus/MCAQuests?utm_source=chatgpt.com

[2] [10] GitHub - IAFEnvoy/IceAndFire-CE: An unofficial fork of Ice And Fire. · GitHub
https://github.com/IAFEnvoy/IceAndFire-CE?utm_source=chatgpt.com

5 8 14 Bounty Customization | Kambrik
https://kambrik.ejekta.io/mods/bountiful/latest/CustomizingBounties?utm_source=chatgpt.com

7 13 Bounty Generation | Kambrik
https://kambrik.ejekta.io/mods/bountiful/advanced/generation?utm_source=chatgpt.com

9 [Bug]: Crash on startup with Ice And Fire: Community Edition (NoClassDefFoundError: IafItemRegistry) · Issue #239 · IAFEnvoy/IceAndFire-CE · GitHub
https://github.com/IAFEnvoy/IceAndFire-CE/issues/239?utm_source=chatgpt.com

11 New Content | IAFEnvoy's Docs
https://docs.iafenvoy.com/docs/mod/ice-and-fire-ce/new-content/?utm_source=chatgpt.com

12 15 Reputation | Kambrik
https://kambrik.ejekta.io/mods/bountiful/general/reputation?utm_source=chatgpt.com

16 Bountiful | Kambrik
https://kambrik.ejekta.io/mods/bountiful/?utm_source=chatgpt.com

17 Kambrik | Kambrik
https://kambrik.ejekta.io/mods/bountiful/1-16/Commands?utm_source=chatgpt.com

18 Bounty Boards | Kambrik
https://kambrik.ejekta.io/mods/bountiful/general/bounty-boards?utm_source=chatgpt.com

19 GitHub - ejektaflex/Bountiful: A Minecraft mod adding bounties for specific items. · GitHub
https://github.com/ejektaflex/Bountiful?utm_source=chatgpt.com